

# Candidate Data Unification Pipeline

## 1. PIPELINE BREAKDOWN

Raw candidate data from **four source types** — **Recruiter CSV, ATS JSON, Resume, and Recruiter Notes** — flows through seven stages. **Adapt** — each source is parsed by a dedicated adapter into a common {field: value} shape, tagged with its source and extraction method. **Normalize** — every field is converted to a canonical format independent of source: phones to E.164, dates to YYYY-MM, country to ISO-3166 alpha-2, skills to a canonical name via a lookup table. **Match** — raw records are grouped by candidate identity using a priority-ordered key: exact email match first, normalized-phone match as fallback; with neither identifier present, the record stands alone rather than being guessed into a merge. **Merge** — for each field in a matched group, conflicting values are resolved by source priority (ATS > CSV > Resume > Notes); priority applies only when sources disagree, and a missing field in a higher-priority source never blocks a lower-priority value. **Confidence** — each field is scored from cross-source agreement and source authority; overall\_confidence is a weighted average across populated fields, weighted toward identity fields (email, phone) since they drive correct matching. **Project** — a runtime config reshapes the canonical record (field selection, renaming, per-field normalization override, provenance/confidence toggling) without altering the underlying merge logic. **Validate** — the projected output is checked against the requested schema; a failing field is set to null rather than invented, and the run never crashes on bad input.

Matching uses indexed lookup by email and phone rather than pairwise comparison, keeping the pipeline roughly linear as candidate volume grows.

## 2. CANONICAL SCHEMA & NORMALIZED FORMATS

Phones are normalized to E.164; dates (experience start/end, education end\_year) to YYYY-MM; country to ISO-3166 alpha-2. Skills are canonicalized through a synonym lookup table (e.g., “JS,” “Java Script,” “javascript” → JavaScript), so the same competency is never represented twice under different spellings.

## 3. MERGE & CONFLICT-RESOLUTION POLICY

Matching uses case-insensitive exact email as the primary key, falling back to normalized phone when email is absent on either side; name is never used as a sole match key, since it produces false-positive merges between unrelated candidates. Field conflicts within a matched group are resolved by fixed source priority — ATS JSON, then Recruiter CSV, then Resume, then Recruiter Notes — reflecting that structured, system-of-record data is more trustworthy than free-text extraction; priority is invoked only on genuine disagreement, and a missing field in a higher-priority source never blocks a lower-priority value. Confidence is derived from corroboration (more independent agreeing sources raise confidence) and source authority. Every field in the final record carries provenance — which source supplied it and how (direct read, regex extraction, or inference) — independent of its confidence score.

## 4. RUNTIME-CONFIGURABLE OUTPUT

The pipeline always builds one full internal canonical record first, regardless of how it will later be displayed. A separate projection layer takes this record plus a runtime config and produces the requested view: selecting a field subset, remapping field names via a “from” path, applying per-field normalization overrides, toggling provenance and confidence visibility, and applying the configured missing-value policy (null, omit, or error). Merge logic and normalization never re-run per config — the config only changes what is shown, not how it was derived. The projected result is validated against the requested schema before being returned.

## 5. EDGE CASES

Two candidates sharing a name with no email or phone are not merged — two separate profiles are created, since name alone is not a reliable identifier. A source with no email or phone at all still has the candidate emitted, with those fields null, never skipped. Inconsistent skill spelling across sources is normalized to one canonical name before merge. A corrupted or empty source file has its adapter return an empty result set, and the pipeline continues using the remaining sources rather than failing. Conflicting emails with no other shared identifier are not auto-merged, producing two profiles rather than guessing which is correct. Explicitly out of scope given the time constraint: fuzzy or ML-based name matching for deduplication beyond exact email/phone keys — a known source of false positives that would need more validation effort than this timeline allows.